Module 2 — FastAPI Basics (Minimum Syntax for AI Engineers)

Learning Objectives

By the end of this module, you will be able to:

- **Install** FastAPI and its production-grade server, Uvicorn.
- **Initialize** a FastAPI application instance to act as your model hub.
- **Implement** the two core HTTP operations used in AI: GET (Health Checks) and POST (Model Inference).
- **Deploy** a local development server to test your model's connectivity.
- **Navigate** automatic documentation to test your endpoints without a frontend.

Concepts and Explanations

1. The Environment and Installation

Before writing code, an AI Engineer must set up a controlled environment. FastAPI is not just a library; it requires a server to "serve" your model.

- **FastAPI [Standard]:** This version includes **Uvicorn**, a high-performance server that handles the actual network communication between the user and your Python code.
- **Why this is needed:** Model inference can be computationally expensive. You need a server that is "Fast" (high performance) to minimize the overhead between the user request and the model calculation.
- **Where it is used:** Every production ML system (like those at **Uber** or **Microsoft**) uses a server like Uvicorn to handle incoming prediction requests.

2. The FastAPI "App" Instance

The "App" is the main object where you register your model's routes.

- **The Instance:** You create an instance of the `FastAPI` class. This object is the "hub" for your service.
- **Why this is needed:** It acts as the brain of your backend service, coordinating how data moves from the internet into your specific Python functions.

3. Paths and Operations (GET vs. POST)

In AI Engineering, we use specific "Operations" for specific tasks.

- **GET (Read):** Used for **Health Checks**. A health check is a simple endpoint that tells a monitoring system if your model service is still running or if the model has crashed.
- **POST (Create/Send):** Used for **Model Inference**. When sending features (like an array of numbers, a text string, or an image), we use POST because it allows us to send a "Request Body"—a package of data that doesn't have to fit inside a URL.
- **Path Operation Decorator:** The code syntax (e.g., `@app.get("/")`) that tells FastAPI which URL path corresponds to which Python function.

4. The Development Server (Uvicorn)

You cannot run a FastAPI file by just clicking "Play" in your IDE like a normal script. You must use a command-line tool called **Uvicorn** to start the service.

- **Reload Flag:** During development, we use `--reload` so the server restarts every time you fix a bug in your model logic.
- **Why this is needed:** It allows you to test your model as if it were live on the internet while you are still writing the code.

Code Examples

Example 1: The AI Service "Health Check" (GET)

This is the simplest possible service. It verifies the API is alive.

```
from fastapi import FastAPI  #

app = FastAPI()  #

@app.get("/health")  #
def check_health():  #
    return {"status": "Model is Ready"}  #
```

Line-by-Line Explanation:

1. **`from fastapi import FastAPI`**: We import the tool that builds the interface.
2. **`app = FastAPI()`**: We create the "App" instance which will hold our model endpoints.
3. **`@app.get("/health")`**: This is a "decorator." It tells the computer: "If someone visits the URL ending in `/health` using a GET request, run the function below".
4. **`def check_health():`**: A standard Python function. In FastAPI, this is called a **Path Operation Function**.
5. **`return {"status": "Model is Ready"}`**: We return a Python dictionary. FastAPI automatically converts this into **JSON** format for the user.

Example 2: Dummy Inference Endpoint (POST)

This simulates receiving data for a model prediction.

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/predict")  #
def get_prediction():  #
    # In a real scenario, your model code would go here
    return {"prediction": "Pass", "confidence": 0.98}  #
```

Line-by-Line Explanation:

1. **`@app.post("/predict")`**: We use **POST** here because inference usually involves sending data (like features) to the model.
2. **`def get_prediction():`**: This function will eventually hold your `model.predict()` logic.
3. **`return {...}`**: The service sends back the dummy result as a JSON response.

Running the Service

To see your code in action, you run this command in your terminal:

```
fastapi dev main.py
```

(Assuming your file is named `main.py`)

- **`fastapi dev`**: Starts the server in development mode with **auto-reload**.
- **The Output**: The terminal will show a link: `http://127.0.0.1:8000`. This is where your service is living on your machine.

Automatic Documentation

FastAPI automatically creates a testing UI for you. While the server is running, go to: `http://127.0.0.1:8000/docs`

- **Swagger UI:** This page lists all your endpoints (`/health` and `/predict`). You can click "Try it out" to test them without writing any client-side code.

Common Mistakes AI Engineers Make

- **Running the file as a script:** Trying to run `python main.py` and wondering why nothing happens. You **must** use `fastapi dev` or `uvicorn` to start the server.
- **Naming Confusion:** Naming the file `fastapi.py`. This will cause a "circular import" error because Python will try to import your file instead of the actual FastAPI library.

- **The "App" Name Mismatch:** If you name your instance `my_model = FastAPI()`, you must run the server using `uvicorn main:my_model`.
- **Forgetting the Slash:** Forgetting the leading `/` in the path (e.g., `@app.get("health")` instead of `@app.get("/health")`). FastAPI requires the forward slash.

Assignment: Create a "Model Identity" API

Task:

1. Create a file named `inference_service.py`.
2. Create a FastAPI app instance.
3. Create a **GET** endpoint at `/info` that returns the name of your favorite ML model and its version (e.g., `{"model": "ResNet50", "version": 1.2}`).
4. Create a **POST** endpoint at `/inference` that returns a dummy dictionary: `{"result": "Label_A"}`.
5. Run the server using `fastapi dev inference_service.py` and verify it works by visiting the `/docs` page.

Mini Project Idea

The "Model Metadata Service": Build a service that acts as a directory for three different models (e.g., "ImageClassifier", "TextSummarizer", "PricePredictor"). Use different **GET** paths for each (e.g., `/models/image`, `/models/text`) to return their specific metadata like "last_trained_date" and "accuracy_score".

Readiness Checklist

Before moving to the next module, ensure you can:

- [ ] Install FastAPI and verify Uvicorn is available.
- [ ] Explain why an AI Engineer uses **POST** for inference instead of GET.
- [ ] Start a development server and see the "Uvicorn running on..." message.
- [ ] Open the `/docs` (Swagger UI) and successfully click "Execute" on an endpoint.
- [ ] Correctly define a "Path Operation Function" and its "Decorator".